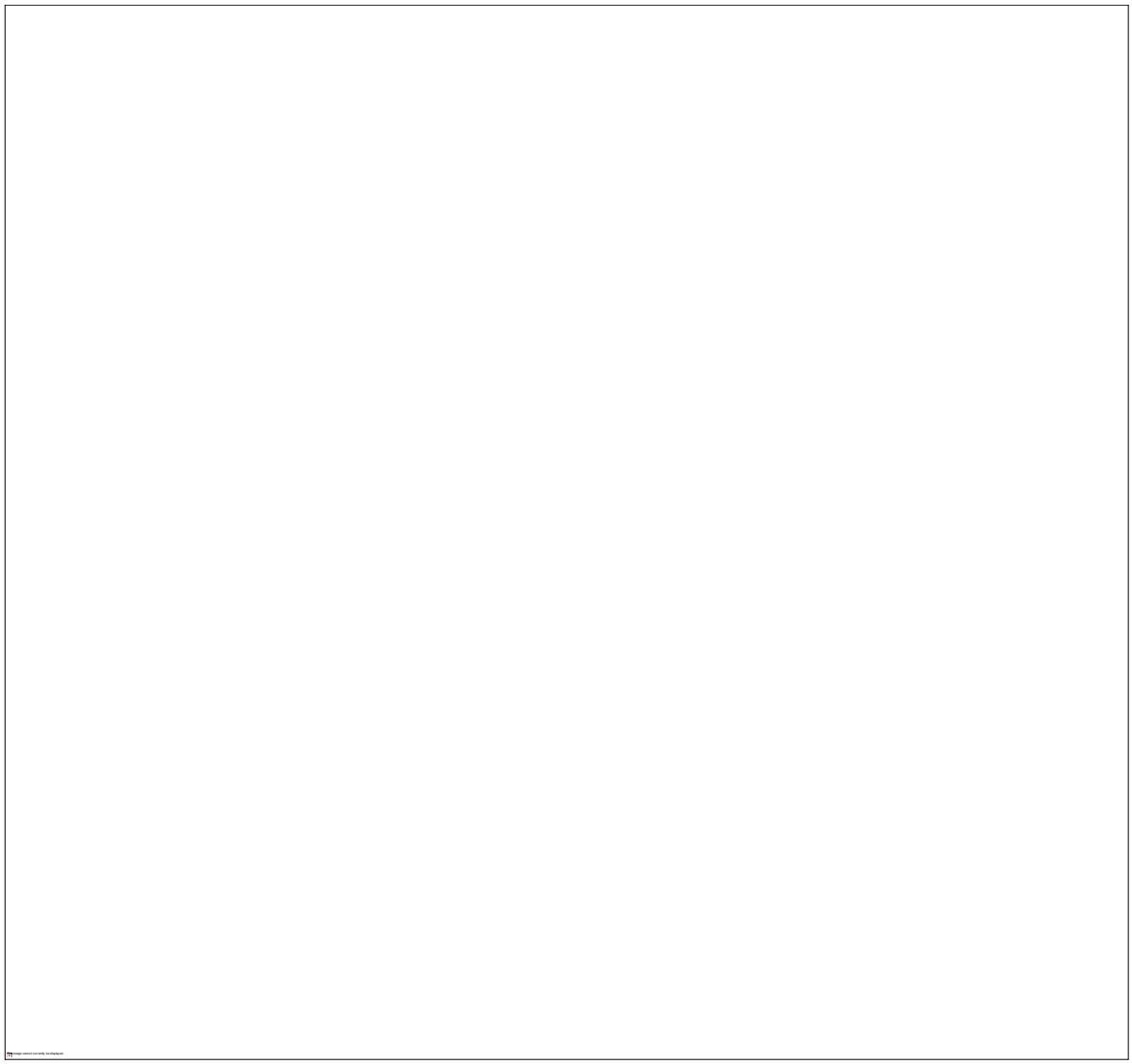
Explore Your Network Configuration

1. **hostname** :This prints the machine's hostname – the name it identifies itself by on the network. It shows up in the shell prompt, in log files, and whenever this machine introduces itself to another one.
   Typical output: ubuntu-devops-vm
   Just one line, nothing more. It's mainly useful for keeping track of which machine you're on when you're regularly logging into several servers over SSH.
2. **hostname -I** :The capital -I flag changes the command completely: instead of the name, it lists every IP address assigned to the machine across all of its active network interfaces, separated by spaces. It's the fastest way to find a device's own address without reading through the longer output of ip addr.
   Typical output: 192.168.1.15
   That address would normally be a private IP handed out by the router. If the machine has more than one active interface wired plus Wi-Fi, or a Docker network more than one address can appear on the same line.
3. **ip addr** :A modern command (from the iproute2 package) that lists every network interface on the machine along with full detail on each: whether it's up or down, its IP address, and its MAC address. On current Linux systems this has mostly replaced the older ifconfig.
   Typical output:
   1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 inet 127.0.0.1/8 scope host lo
   2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 link/ether 08:00:27:4e:2a:91 brd ff:ff:ff:ff:ff:ff inet 192.168.1.15/24 brd 192.168.1.255 scope global eth0
   lo is the loopback interface (127.0.0.1) the machine uses to talk to itself; eth0 is the actual network card. UP means the interface is active, link/ether is its MAC address, and inet 192.168.1.15/24 is the assigned IP address together with its subnet mask (/24 is the same as 255.255.255.0).



4. **ip route :** This shows the machine's routing table the rules it follows to decide where outgoing traffic should go, depending on the destination.
   Typical output: default via 192.168.1.1 dev eth0 proto dhcp metric 100 192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.15
   The default via 192.168.1.1 line is the default gateway from Task 1 – anything that isn't on the local network gets sent there. The second line says traffic to 192.168.1.0/24 (the local subnet) goes straight out of eth0, with no gateway needed, because those devices are already on the same network.

5. **ifconfig :** The older, traditional way of viewing (and configuring) network interfaces, from the net-tools package. Most current distros consider it deprecated in favor of ip addr, and it sometimes has to be installed separately (sudo apt install net-tools), but it's still extremely common in day-to-day use.

Typical output: eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500 inet 192.168.1.15 netmask 255.255.255.0 broadcast 192.168.1.255 ether 08:00:27:4e:2a:91 txqueuelen 1000 (Ethernet) RX packets 48213 bytes 51234112 (51.2 MB) TX packets 30122 bytes 4213556 (4.2 MB)

It's largely the same information as ip addr IP address, netmask, MAC address just laid out differently, plus RX/TX counters showing how many packets and bytes the interface has received and sent since it came up.

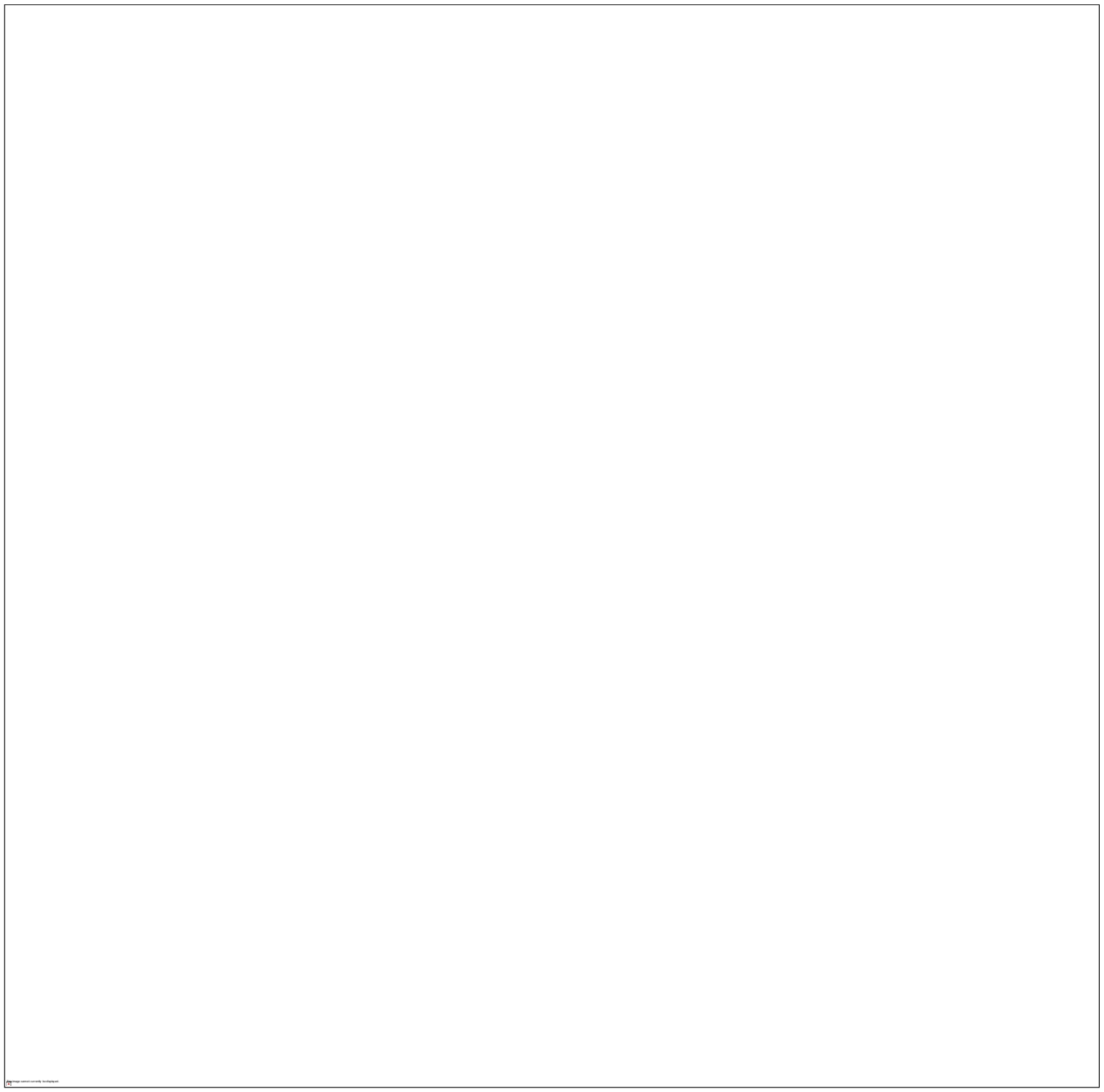
6. **nmcli device status :** nmcli is the command-line interface for NetworkManager. This particular command lists every device NetworkManager knows about, what type it is, whether it's connected, and which saved connection profile it's using. It's common on desktop Linux distributions like Ubuntu Desktop; many minimal servers don't run NetworkManager at all, in which case this command simply isn't installed.

   Typical output: DEVICE TYPE eth0 STATE ethernet connected lo CONNECTION loopback connected (externally) lo wlan0 wifi Wired connection 1 disconnected -A quick, at-a-glance summary of what's connected and what isn't. If the command isn't found, that just means NetworkManager isn't installed on that machine – ip addr or ifconfig cover the same ground.

7. **cat /etc/resolv.conf:** This one isn't a networking command at all – it just prints a config file. That file tells the system which DNS servers to use when turning domain names into IP addresses .

   Typical output: nameserver 127.0.0.53 options edns0 trust-ad search localdomain

   Each nameserver line is a DNS server being queried. On current Ubuntu systems it's often 127.0.0.53, a local stub resolver (systemd-resolved) that forwards requests on to the real DNS servers behind the scenes; on other setups this might instead show the router's address directly, or a public resolver like 8.8.8.8 (Google) or 1.1.1.1
   (Cloudflare).



8. ping google.com: This sends small ICMP “echo request” packets to google.com and waits for replies, which tests two things at once: that DNS is resolving the domain correctly, and that there's an actual working path to reach it. On Linux it keeps running until stopped with Ctrl+C, or until it hits a fixed count if given the -c flag.

Typical output: PING google.com (142.250.183.14) 56(84) bytes of data. 64 bytes from 142.250.183.14: icmp_seq=1 ttl=115 time=18.4 ms 64 bytes from 142.250.183.14: icmp_seq=2 ttl=115 time=17.9 ms--- google.com ping statistics --2 packets transmitted, 2 received, 0% packet loss, time 1001ms rtt min/avg/max/mdev = 17.912/18.156/18.401/0.245 ms

The first line already confirms DNS worked, since google.com resolved to a real IP address. Each 64 bytes from … line is one reply, and time= is the round-trip time in milliseconds – lower is better. 0% packet loss at the bottom means every packet sent got a reply. If it fails instead,

Temporary failure in name resolution points to a DNS problem, while Network is unreachable or requests simply timing out point to a routing or connectivity
problem.

